

UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

Informe detallado de pruebas unitarias de telemedicina

Resultados, alcance, cambios y evidencias de ejecución

Sistema de telemedicina con triaje mediante modelos de inteligencia artificial

Cochabamba - Bolivia

2026

Resultados de la ejecución

Las ejecuciones finales verificaron 69 casos únicos: 62 en siete suites de Jest y 7 en unittest de Python. Todos cumplieron sus aserciones, sin fallidos ni omitidos en el resultado final. Este resultado comprende validaciones, decisiones locales, preparación de contenido y servicios sustituidos; no representa la aprobación de módulos completos.

La primera ejecución de Python registró siete errores de preparación por la resolución del módulo dotenv al instalar los sustitutos. Se conservó ese intento, se corrigió únicamente el mecanismo de sustitución y se repitieron los siete casos, que aprobaron. Los siete intentos iniciales no se suman como nuevos casos.

Tabla 1. Resultados finales por grupo de ejecución

Grupo	Ejecutor	Casos aprobados	Fallidos / omitidos
Gestión de usuarios	Jest	30	0 / 0
Triaje mediante inteligencia artificial	Jest	7	0 / 0
Triaje del servicio Python	unittest	7	0 / 0
Videoconsultas	Jest	7	0 / 0
Historial y recetas	Jest	9	0 / 0
Pagos	Jest	4	0 / 0
Reportes	Jest	5	0 / 0

Fuente: Elaboración propia, 2026

Herramientas y configuración

Tabla 2. Versiones instaladas verificadas

Herramienta o tecnología	Versión	Uso
Jest	29.7.0	Ejecuta las pruebas de las funciones JavaScript y TypeScript.
jest-expo	57.0.5	Adapta Jest al entorno de la aplicación Expo.
Python y unittest	3.12.14	Ejecutan las pruebas aisladas del servicio de análisis; unittest forma parte de Python.
Node.js y npm	24.15.0 / 11.12.1	Ejecutan las herramientas y administran las dependencias.
TypeScript	6.0.3	Permite comprobar los tipos del proyecto.
Expo	57.0.19	Es una tecnología de la aplicación; no es el ejecutor de pruebas.

Fuente: Elaboración propia, 2026

Jest utiliza el preset jest-expo, los patrones de testMatch y las extensiones ts, tsx, js, jsx y json de jest.config.js. Las ejecuciones seleccionaron archivos explícitos con --runTestsByPath, desactivaron el modo de observación y se ejecutaron en un solo proceso. No se ejecutaron indiscriminadamente otros scripts del repositorio.

Las pruebas de pagos sustituyen el módulo lib/supabase antes de importarlo. Las pruebas Python sustituyen dotenv.load_dotenv, urllib.request.urlopen, os.getenv y la escritura de errores. No se cargó el archivo de variables de entorno ni se realizaron solicitudes reales al proveedor. El servicio Python se probó sin iniciar FastAPI.

La comprobación estática con tsc --noEmit --pretty false finalizó con código 0, sin diagnósticos. git diff --check no detectó errores de espacios; sus avisos sobre LF/CRLF no son fallos de las pruebas. Estas comprobaciones complementarias no se suman a los 69 casos.

Identificación de las ejecuciones

Fecha: 30 de septiembre de 2026. Entorno: Windows, PowerShell y zona America/La_Paz, UTC-04:00. Los tiempos de la tabla corresponden al inicio local de cada ejecución final; los registros JSON conservan inicio y fin en UTC.

Referencia del repositorio: 355b1d9ddb8508f811fa5a9e419eba16eb0919e2. Había cambios locales antes de ampliar las pruebas, por lo que el identificador del commit no representa por sí solo la versión ejecutada. Se conservaron inventarios por ejecución, fuentes copiadas y hashes de respaldo.

Tabla 3. Archivos y fecha de las ejecuciones finales

Carpeta de evidencia	Inicio UTC-04:00	Archivos ejecutados
usuarios	30/09/2026 16:45:56	__tests__/loginValidation.test.ts __tests__/userValidation.test.ts
triaje	30/09/2026 16:52:21	__tests__/triageWorkflow.test.ts
triaje-python	30/09/2026 16:54:09	ai-engine/test_unit_llm.py
videoconsultas	30/09/2026 16:53:36	__tests__/consultationRules.test.ts
historial-recetas	30/09/2026 16:54:13	__tests__/medicalRecord.test.ts
pagos	30/09/2026 16:54:35	__tests__/paymentSimulation.test.ts
reportes	30/09/2026 16:54:41	__tests__/reportStats.test.ts

Fuente: Elaboración propia, 2026

Carpeta raíz de evidencia: evidencias/pruebas-completas. Cada subcarpeta contiene ejecucion.json con comando, versiones, fecha y referencia local; resultado.json con estados; salida.log con la salida textual; fuentes con las copias utilizadas; archivos.json, alcance.md y capturas.md. La carpeta triaje-python conserva también intento-inicial.json e intento-inicial.log.

Comandos registrados

usuarios

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/loginValidation.test.ts __tests__/userValidation.test.ts --json --outputFile=evidencias/pruebas-completas/usuarios/resultado.json
```

triaje

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/triageWorkflow.test.ts --json --outputFile=evidencias/pruebas-completas/triaje/resultado.json
```

triaje-python

```
"C:\Users\inutil\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe" ai-engine/test_unit_llm.py
```

videoconsultas

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/consultationRules.test.ts --json --outputFile=evidencias/pruebas-completas/videoconsultas/resultado.json
```

historial-recetas

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/medicalRecord.test.ts --json --outputFile=evidencias/pruebas-completas/historial-recetas/resultado.json
```

pagos

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/paymentSimulation.test.ts --json --outputFile=evidencias/pruebas-completas/pagos/resultado.json
```

reportes

```
node node_modules/jest/bin/jest.js --runInBand --watch=false --verbose --runTestsByPath __tests__/reportStats.test.ts --json --outputFile=evidencias/pruebas-completas/reportes/resultado.json
```

Los comandos anteriores documentan la ejecución ya realizada. La guía de capturas propone ejecuciones adicionales que no sobrescriben esos resultados históricos.

Módulos y casos de uso revisados

Gestión de usuarios

Se evaluaron cuatro validadores locales. Autenticación, alta de cuentas, asignación de roles y persistencia de perfiles requieren integración con Supabase.

Funciones probadas: validateLoginFields, validateRegistrationFields, validatePatientProfileFields, validateDoctorProfileFields.

Casos de uso con alcance unitario: Iniciar sesión, Registrar usuario, Editar perfil del paciente, Editar perfil del médico.

Triaje mediante inteligencia artificial

Se comprobaron preparación de texto y filtro de especialidades. No se evaluaron micrófono, interfaz, traducción real ni exactitud clínica.

Funciones probadas: normalizeSymptomText, buildSymptomReport, filterTriagesBySpecialty, detect_heuristic_specialty, analyze_symptoms_with_gemini, transcribe_audio_only.

Casos de uso con alcance unitario: Capturar sintomatología en Quechuañol, Clasificar urgencia y derivar al paciente, Procesar y traducir sintomatología mediante IA.

Videoconsultas

Se evaluaron decisiones locales de la sala de espera. No se probó conexión, audio, video ni actualización real de disponibilidad. El fallo manual informado permanece pendiente de reproducción y corrección.

Funciones probadas: isClosedTriage, isDoctorReadyForCall.

Casos de uso con alcance unitario: Visualizar sala de espera.

Historial y recetas

Se comprobaron campos obligatorios, preparación del prediagnóstico y fragmentos del HTML de la receta. No se probó guardar el historial, generar el archivo PDF, compartirlo ni verificar su contenido clínico.

Funciones probadas: hasClinicalRecordFields, prepareTriageView, generatePrescriptionHtml.

Casos de uso con alcance unitario: Registrar historial clínico, Visualizar prediagnóstico de IA, Autogenerar receta PDF.

Pagos

Solo se probó la función de simulación con Supabase sustituido. La generación de un QR bancario y la verificación de transacciones reales no están implementadas.

Funciones probadas: markSimulatedPaymentWaiting.

Casos de uso con alcance unitario: Verificar transacción.

Reportes

Existe la pantalla y su cálculo estadístico. Se evaluaron contadores y estimaciones, no la consulta a Supabase. No se implementaron los filtros por fecha y exportación previstos por el caso de uso.

Funciones probadas: calculateReportStats.

Casos de uso con alcance unitario: Generar reportes estadísticos.

Cambios y repetición de pruebas

Para permitir pruebas aisladas, se extrajeron reglas existentes de sus componentes y se reemplazaron los bloques originales por llamadas a esas funciones. Se conservaron sus condiciones y resultados actuales, incluidas las limitaciones. No se corrigieron H-01 ni H-02 ni se cambiaron expectativas para obtener resultados aprobados.

Tabla 4. Archivos creados y pantallas adaptadas en esta ampliación

Grupo	Archivos nuevos	Archivos modificados
triaje	__tests__/triageWorkflow.test.ts utils/triageWorkflow.ts	components/SymptomChatbot.tsx app/(doctor)/dashboard.tsx
triaje-python	ai-engine/test_unit_llm.py	Ninguna; llm_service.py se conserva.
videoconsultas	__tests__/consultationRules.test.ts utils/consultationRules.ts	app/(patient)/waiting-room.tsx
historial-recetas	__tests__/medicalRecord.test.ts utils/medicalRecord.ts utils/triagePresentation.ts	app/(doctor)/medical-record.tsx components/TriageEvaluation.tsx
pagos	__tests__/paymentSimulation.test.ts services/paymentSimulation.ts	app/(patient)/payment.tsx
reportes	__tests__/reportStats.test.ts utils/reportStats.ts	app/(admin)/reports.tsx

Fuente: Elaboración propia, 2026

Se creó scripts/record-unit-tests.cjs para ejecutar archivos seleccionados y guardar resultados. Se crearon también los archivos de evidencia y los dos documentos independientes. Las fuentes de usuarios y sus dos archivos de prueba ya existían al comenzar esta ampliación y se volvieron a ejecutar sin cambiar sus expectativas. Las modificaciones previas en registro, perfiles, package.json, package-lock.json y tsconfig.json no se atribuyen a esta ampliación.

Error de preparación y corrección mínima

El intento inicial del servicio Python tuvo siete errores antes de ejecutar las aserciones: ModuleNotFoundError para dotenv. La restauración de sys.modules al terminar la sustitución de dotenv provocó que patch con una ruta textual intentara importar llm_service de nuevo. Se cambió a patch.object sobre el módulo ya cargado, manteniendo sustituidas la red y la lectura de configuración. No se modificó llm_service.py ni las expectativas de los casos.

La repetición final de los siete casos Python aprobó sin errores. La evidencia del fallo y de la repetición se conserva en triaje-python. La ejecución de videoconsultas quedó registrada entre el intento inicial de Python y su repetición final; los tiempos de cada registro permiten reconstruir esa secuencia.

Hallazgos funcionales pendientes

H-01. El registro acepta un nombre formado solo por espacios. La validación local no lo considera vacío. Estado: pendiente de corrección y nueva comprobación. PU-21.

H-02. El perfil del paciente acepta teléfono y dirección formados solo por espacios. Estado: pendiente de corrección y nueva comprobación. PU-25.

H-03. El filtro de especialidad conserva un triaje sin especialidad, incluso al seleccionar Pediatría. No confirma una derivación adecuada. Estado: pendiente de corrección y nueva comprobación. PU-37.

H-04. La decisión local de habilitar la llamada acepta un objeto de cita cancelada porque no comprueba su estado. No se reprodujo una conexión real. Estado: pendiente de corrección y nueva comprobación. PU-51.

H-05. La validación local del historial acepta diagnóstico y tratamiento compuestos solo por espacios. Estado: pendiente de corrección y nueva comprobación. PU-55.

H-06. La preparación del prediagnóstico utiliza urgencia Medium ante un objeto vacío; si no hay triaje, devuelve datos de demostración y urgencia Critical. Estos valores no representan una evaluación clínica. Estado: pendiente de corrección y nueva comprobación. PU-57, PU-58.

H-07. La simulación de pago no propaga los errores: ignora los errores devueltos y solo registra las excepciones. La pantalla conserva su posterior indicación de éxito; este último recorrido no se ejecutó en interfaz. Estado: pendiente de corrección y nueva comprobación. PU-63, PU-64.

H-08. El contador denominado completedTriages incluye estados waiting e in_progress. Los ingresos se estiman multiplicando ciertos registros por 50; no provienen de cobros bancarios. Estado: pendiente de corrección y nueva comprobación. PU-67.

H-09. El servicio de análisis devuelve una respuesta incompleta sin exigir urgency_level ni standardized_symptoms. La prueba se limita al servicio y no evalúa los valores de respaldo del endpoint FastAPI. Estado: pendiente de corrección y nueva comprobación. PU-41.

Actividades que no son pruebas unitarias ejecutadas

Asignar roles y especialidades. Requiere integración. La asignación real, autorización y persistencia de roles y especialidades no se ejecutaron. Las validaciones del registro no acreditan estos cambios.

Controlar disponibilidad médica. Pendiente. Existe el control de disponibilidad. El fallo manual informado —el botón no cambia el estado— no se reprodujo en esta ejecución ni se declara corregido. Se requiere comprobar interfaz, sesión y actualización en Supabase.

Transmitir videoconsulta. Requiere integración. La aplicación utiliza el componente de videollamada, pero no se estableció una conexión entre médico y paciente ni se comprobaron audio y video.

Generar código QR de pago. No implementado. La pantalla muestra un icono QR y una simulación. No se identificó generación de un código bancario verificable.

Autenticar usuarios, crear cuentas, guardar perfiles y comprobar permisos requieren integración. En triaje, la calidad de la traducción, la clasificación de urgencia y la derivación necesitan casos clínicos y revisión profesional; las respuestas simuladas no son una validación médica. El procesamiento real de voz requiere integración.

Guardar y recuperar el historial, generar y compartir un PDF real requieren integración y revisión manual. En pagos, la verificación bancaria está sin implementar. En reportes, falta implementar filtros por fecha y exportación; la consulta real de datos permanece sin comprobar. No se midió cobertura de líneas, ramas o seguridad del sistema.

Detalle de los casos ejecutados

Las tablas presentan los diez campos de cada caso en orientación vertical, agrupando tres identificadores por tabla para conservar la legibilidad. «Resultado obtenido» expresa las comprobaciones respaldadas por las aserciones aprobadas; no se registraron objetos adicionales no comprobados por los tests. Las rutas de evidencia se resuelven dentro de evidencias/pruebas-completas.

Los registros de paciente usados en usuarios tienen correo paciente@example.test, contraseña ficticia Ficticia123!, nombre Persona de Prueba, identidad TEST-001, teléfono 00000000 y Dirección ficticia. Los de médico agregan especialidad-ficticia y un objeto de documento titulo-ficticio.pdf que no se abre ni se carga. Los literales de los cinco casos originales también son datos de prueba, no credenciales verificadas.

Tabla 5. Detalle de PU-01 a PU-03

Identificador	PU-01	PU-02	PU-03
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Iniciar sesión	Iniciar sesión	Iniciar sesión
Función	validateLoginFields	validateLoginFields	validateLoginFields
Escenario	Ambos campos del inicio de sesión están vacíos.	La contraseña está vacía.	El correo está vacío.
Datos de prueba	email="", password=""	email=correo ficticio, password=""	email="", password=clave ficticia
Resultado esperado	La validación rechaza los datos y solicita ingresar el correo y la contraseña.	La validación rechaza los datos y solicita ingresar la contraseña.	La validación rechaza los datos y solicita ingresar el correo electrónico.
Resultado obtenido	isValid=false; mensaje «Por favor ingresa tu correo y contraseña.».	isValid=false; mensaje «Por favor ingresa tu contraseña.».	isValid=false; mensaje «Por favor ingresa tu correo electrónico.».
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-01 (caso original 1)	usuarios/resultado.json y salida.log; PU-02 (caso original 2)	usuarios/resultado.json y salida.log; PU-03 (caso original 3)

Fuente: Elaboración propia, 2026

Tabla 6. Detalle de PU-04 a PU-06

Identificador	PU-04	PU-05	PU-06
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Iniciar sesión	Iniciar sesión	Iniciar sesión
Función	validateLoginFields	validateLoginFields	validateLoginFields
Escenario	El correo contiene únicamente espacios.	El correo y la contraseña están completos.	Los dos argumentos se reciben sin un valor definido.
Datos de prueba	email=cinco espacios, password=clave ficticia	email=correo ficticio, password=clave ficticia	email=undefined, password=undefined
Resultado esperado	La validación rechaza el correo de solo espacios y solicita ingresar el correo electrónico.	Los datos superan la validación local y no se devuelve un mensaje de error. No se comprueba la autenticación.	La validación rechaza las entradas y solicita ingresar el correo y la contraseña.
Resultado obtenido	isValid=false; mensaje «Por favor ingresa tu correo electrónico.».	isValid=true, sin mensaje de error.	isValid=false; mensaje «Por favor ingresa tu correo y contraseña.».
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-04 (caso original 4)	usuarios/resultado.json y salida.log; PU-05 (caso original 5)	usuarios/resultado.json y salida.log; PU-06

Fuente: Elaboración propia, 2026

Tabla 7. Detalle de PU-07 a PU-09

Identificador	PU-07	PU-08	PU-09
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Iniciar sesión	Iniciar sesión	Iniciar sesión
Función	validateLoginFields	validateLoginFields	validateLoginFields
Escenario	Los dos argumentos se reciben con valor nulo.	El correo tiene espacios al principio y al final.	La contraseña contiene únicamente espacios.
Datos de prueba	email=null, password=null	email="paciente@example.test ", password=clave ficticia	email=correo ficticio, password=tres espacios
Resultado esperado	La validación rechaza las entradas y solicita ingresar el correo y la contraseña.	Los datos superan la validación local y no se devuelve un mensaje de error.	Los datos superan la validación local porque la contraseña tiene caracteres. No se recortan sus espacios ni se comprueba la autenticación.
Resultado obtenido	isValid=false; mensaje «Por favor ingresa tu correo y contraseña.».	isValid=true, sin mensaje de error.	isValid=true, sin mensaje de error.
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-07	usuarios/resultado.json y salida.log; PU-08	usuarios/resultado.json y salida.log; PU-09

Fuente: Elaboración propia, 2026

Tabla 8. Detalle de PU-10 a PU-12

Identificador	PU-10	PU-11	PU-12
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Registrar usuario	Registrar usuario	Registrar usuario
Función	validateRegistrationFields	validateRegistrationFields	validateRegistrationFields
Escenario	El correo está vacío.	La contraseña está vacía.	El nombre completo está vacío.
Datos de prueba	Paciente base con email=""	Paciente base con password=""	Paciente base con fullName=""
Resultado esperado	La validación rechaza el registro y solicita completar todos los campos comunes.	La validación rechaza el registro y solicita completar todos los campos comunes.	La validación rechaza el registro y solicita completar todos los campos comunes.
Resultado obtenido	isValid=false; mensaje «Por favor completa todos los campos comunes.».	isValid=false; mensaje «Por favor completa todos los campos comunes.».	isValid=false; mensaje «Por favor completa todos los campos comunes.».
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-10	usuarios/resultado.json y salida.log; PU-11	usuarios/resultado.json y salida.log; PU-12

Fuente: Elaboración propia, 2026

Tabla 9. Detalle de PU-13 a PU-15

Identificador	PU-13	PU-14	PU-15
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Registrar usuario	Registrar usuario	Registrar usuario
Función	validateRegistrationFields	validateRegistrationFields	validateRegistrationFields
Escenario	El carnet de identidad está vacío.	El teléfono está vacío.	La dirección está vacía.
Datos de prueba	Paciente base con identityCard=""	Paciente base con phoneNumber=""	Paciente base con address=""
Resultado esperado	La validación rechaza el registro y solicita completar todos los campos comunes.	La validación rechaza el registro y solicita completar todos los campos comunes.	La validación rechaza el registro y solicita completar todos los campos comunes.
Resultado obtenido	isValid=false; mensaje «Por favor completa todos los campos comunes.».	isValid=false; mensaje «Por favor completa todos los campos comunes.».	isValid=false; mensaje «Por favor completa todos los campos comunes.».
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-13	usuarios/resultado.json y salida.log; PU-14	usuarios/resultado.json y salida.log; PU-15

Fuente: Elaboración propia, 2026

Tabla 10. Detalle de PU-16 a PU-18

Identificador	PU-16	PU-17	PU-18
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Registrar usuario	Registrar usuario	Registrar usuario
Función	validateRegistrationFields	validateRegistrationFields	validateRegistrationFields
Escenario	El paciente completa sus datos sin título ni especialidad.	El médico no tiene una especialidad seleccionada.	El médico no presenta un documento de título.
Datos de prueba	Paciente base sin especialidad ni documento	Médico base con selectedSpecialty=""	Médico base con document=null
Resultado esperado	El registro del paciente supera la validación local sin requerir datos exclusivos del médico.	La validación rechaza los datos y solicita seleccionar una especialidad y adjuntar el título.	La validación rechaza los datos y solicita seleccionar una especialidad y adjuntar el título.
Resultado obtenido	isValid=true, sin mensaje de error.	isValid=false; mensaje «Por favor selecciona una especialidad y adjunta tu título.».	isValid=false; mensaje «Por favor selecciona una especialidad y adjunta tu título.».
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-16	usuarios/resultado.json y salida.log; PU-17	usuarios/resultado.json y salida.log; PU-18

Fuente: Elaboración propia, 2026

Tabla 11. Detalle de PU-19 a PU-21

Identificador	PU-19	PU-20	PU-21
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Registrar usuario	Registrar usuario	Registrar usuario
Función	validateRegistrationFields	validateRegistrationFields	validateRegistrationFields
Escenario	El médico tiene completos sus datos y requisitos.	Faltan un campo común y el título del médico.	El nombre del paciente contiene únicamente espacios.
Datos de prueba	Médico base	Médico base con email="" y document=null	Paciente base con fullName=tres espacios
Resultado esperado	El registro médico supera la validación local. No se crea una cuenta ni se carga el documento.	La validación rechaza los datos y muestra primero la solicitud de completar los campos comunes.	El nombre de solo espacios supera la validación local y no genera un mensaje de error. Se reproduce el comportamiento existente.
Resultado obtenido	isValid=true, sin mensaje de error.	isValid=false; mensaje «Por favor completa todos los campos comunes.».	isValid=true, sin mensaje de error.
Estado	Éxito	Éxito	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional
Evidencia	usuarios/resultado.json y salida.log; PU-19	usuarios/resultado.json y salida.log; PU-20	usuarios/resultado.json y salida.log; PU-21

Fuente: Elaboración propia, 2026

Tabla 12. Detalle de PU-22 a PU-24

Identificador	PU-22	PU-23	PU-24
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Editar perfil del paciente	Editar perfil del paciente	Editar perfil del paciente
Función	validatePatientProfileFields	validatePatientProfileFields	validatePatientProfileFields
Escenario	El teléfono del paciente está vacío.	La dirección del paciente está vacía.	El teléfono y la dirección del paciente tienen contenido.
Datos de prueba	phoneNumber="", address=dirección ficticia	phoneNumber="00000000" , address=""	phoneNumber="00000000" , address=dirección ficticia
Resultado esperado	La validación rechaza los datos y solicita completar el teléfono y la dirección.	La validación rechaza los datos y solicita completar el teléfono y la dirección.	Los datos superan la validación local y no se devuelve un mensaje de error.
Resultado obtenido	isValid=false; mensaje «Por favor completa el número de teléfono y la dirección.».	isValid=false; mensaje «Por favor completa el número de teléfono y la dirección.».	isValid=true, sin mensaje de error.
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-22	usuarios/resultado.json y salida.log; PU-23	usuarios/resultado.json y salida.log; PU-24

Fuente: Elaboración propia, 2026

Tabla 13. Detalle de PU-25 a PU-27

Identificador	PU-25	PU-26	PU-27
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Editar perfil del paciente	Editar perfil del médico	Editar perfil del médico
Función	validatePatientProfileFields	validateDoctorProfileFields	validateDoctorProfileFields
Escenario	El teléfono y la dirección contienen únicamente espacios.	El nombre del médico está vacío.	El celular del médico está vacío.
Datos de prueba	phoneNumber=tres espacios, address=tres espacios	fullName="", phoneNumber="00000000"	fullName="Médico de Prueba", phoneNumber=""
Resultado esperado	Ambos campos superan la validación local y no generan un mensaje de error. Se reproduce el comportamiento existente.	La validación rechaza los datos e informa que el nombre y el celular no pueden estar vacíos.	La validación rechaza los datos e informa que el nombre y el celular no pueden estar vacíos.
Resultado obtenido	isValid=true, sin mensaje de error.	isValid=false; mensaje «El nombre y el celular no pueden estar vacíos».	isValid=false; mensaje «El nombre y el celular no pueden estar vacíos».
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-25	usuarios/resultado.json y salida.log; PU-26	usuarios/resultado.json y salida.log; PU-27

Fuente: Elaboración propia, 2026

Tabla 14. Detalle de PU-28 a PU-30

Identificador	PU-28	PU-29	PU-30
Módulo	Gestión de usuarios	Gestión de usuarios	Gestión de usuarios
Caso de uso	Editar perfil del médico	Editar perfil del médico	Editar perfil del médico
Función	validateDoctorProfileFields	validateDoctorProfileFields	validateDoctorProfileFields
Escenario	El nombre del médico contiene únicamente espacios.	El celular del médico contiene únicamente espacios.	El nombre y el celular tienen espacios exteriores.
Datos de prueba	fullName=tres espacios, phoneNumber="00000000"	fullName="Médico de Prueba", phoneNumber=tres espacios	fullName=" Médico de Prueba ", phoneNumber=" 00000000 "
Resultado esperado	La validación rechaza el nombre de solo espacios e informa que el nombre y el celular no pueden estar vacíos.	La validación rechaza el celular de solo espacios e informa que el nombre y el celular no pueden estar vacíos.	Los datos superan la validación local porque ambos campos contienen información además de los espacios.
Resultado obtenido	isValid=false; mensaje «El nombre y el celular no pueden estar vacíos».	isValid=false; mensaje «El nombre y el celular no pueden estar vacíos».	isValid=true, sin mensaje de error.
Estado	Éxito	Éxito	Éxito
Evidencia	usuarios/resultado.json y salida.log; PU-28	usuarios/resultado.json y salida.log; PU-29	usuarios/resultado.json y salida.log; PU-30

Fuente: Elaboración propia, 2026

Tabla 15. Detalle de PU-31 a PU-33

Identificador	PU-31	PU-32	PU-33
Módulo	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial
Caso de uso	Capturar sintomatología en Quechuañol	Capturar sintomatología en Quechuañol	Capturar sintomatología en Quechuañol
Función	normalizeSymptomText	normalizeSymptomText	buildSymptomReport
Escenario	El texto de síntomas contiene solo espacios.	El relato tiene espacios exteriores.	Se completa el relato, su duración y el uso de medicamentos.
Datos de prueba	Tres espacios.	« uma nanay ».	Síntomas: «uma nanay»; tiempo: «dos horas»; medicamentos: «ninguno».
Resultado esperado	Se obtiene una cadena vacía.	Se obtiene «uma nanay», conservando las palabras del relato.	Se obtienen tres líneas: «Síntomas principales: uma nanay», «Tiempo/ Intensidad: dos horas» y «Medicamentos previos: ninguno».
Resultado obtenido	Cadena vacía.	Texto «uma nanay».	Las tres líneas coincidieron exactamente con las esperadas.
Estado	Éxito	Éxito	Éxito
Evidencia	triaje/resultado.json y salida.log; PU-31	triaje/resultado.json y salida.log; PU-32	triaje/resultado.json y salida.log; PU-33

Fuente: Elaboración propia, 2026

Tabla 16. Detalle de PU-34 a PU-36

Identificador	PU-34	PU-35	PU-36
Módulo	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial
Caso de uso	Clasificar urgencia y derivar al paciente	Clasificar urgencia y derivar al paciente	Clasificar urgencia y derivar al paciente
Función	filterTriagesBySpecialty	filterTriagesBySpecialty	filterTriagesBySpecialty
Escenario	La especialidad tiene diferencias de tildes y mayúsculas.	El médico pertenece a Medicina General.	El filtro por especialidad está desactivado.
Datos de prueba	Cola: « PEDIATRIA » y «Cardiología»; médico: «Pediatria»; filtro activo.	La misma cola ficticia de dos especialidades; médico: «Medicina General»; filtro activo.	Cola de Pediatría y Cardiología; médico: «Pediatria»; filtro inactivo.
Resultado esperado	Se conserva únicamente el registro con especialidad « PEDIATRIA ».	Se conservan los dos registros de la cola.	Se conservan los dos registros de la cola.
Resultado obtenido	Lista con el primer registro de la cola.	Lista original de dos registros.	Lista original de dos registros.
Estado	Éxito	Éxito	Éxito
Evidencia	triaje/resultado.json y salida.log; PU-34	triaje/resultado.json y salida.log; PU-35	triaje/resultado.json y salida.log; PU-36

Fuente: Elaboración propia, 2026

Tabla 17. Detalle de PU-37 a PU-39

Identificador	PU-37	PU-38	PU-39
Módulo	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial
Caso de uso	Clasificar urgencia y derivar al paciente	Clasificar urgencia y derivar al paciente	Clasificar urgencia y derivar al paciente
Función	filterTriagesBySpecialty	detect_heuristic_specialty	analyze_symptoms_with_g emini
Escenario	Un triaje no incluye especialidad recomendada.	El relato contiene una referencia infantil y dolor de pecho.	No está configurado el acceso al proveedor.
Datos de prueba	Cola con un objeto vacío; médico: «Pediatria»; filtro activo.	«Mi wawa tiene dolor de pecho».	Relato «uma nanay»; lectura de configuración simulada como ausente.
Resultado esperado	El registro sin especialidad permanece en la cola. Se reproduce la regla existente.	La regla devuelve «Pediatria». Esto no acredita que la derivación sea clínicamente adecuada.	Se obtiene especialidad «Neurología», urgencia «Medium» y el texto original sin traducir.
Resultado obtenido	Lista con el objeto sin especialidad.	Especialidad Pediatria; ninguna llamada de red.	Neurología; Medium; texto «uma nanay»; ninguna llamada de red.
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito
Evidencia	traje/resultado.json y salida.log; PU-37	traje-python/resultado.json y salida.log; PU-38	traje-python/resultado.json y salida.log; PU-39

Fuente: Elaboración propia, 2026

Tabla 18. Detalle de PU-40 a PU-42

Identificador	PU-40	PU-41	PU-42
Módulo	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial	Triaje mediante inteligencia artificial
Caso de uso	Procesar y traducir sintomatología mediante IA	Procesar y traducir sintomatología mediante IA	Procesar y traducir sintomatología mediante IA
Función	analyze_symptoms_with_g emini	analyze_symptoms_with_g emini	analyze_symptoms_with_g emini
Escenario	El proveedor simulado devuelve un objeto completo dentro de un bloque de texto.	El proveedor simulado omite campos del análisis.	El proveedor simulado devuelve texto sin un objeto JSON.
Datos de prueba	Relato ficticio; JSON simulado: idioma Español, texto «Texto ficticio», urgencia Low, recomendación «Revisión ficticia», especialidad Medicina General.	JSON con solo recommended_specialty igual a «Medicina General».	«respuesta ficticia no estructurada»; relato «relato ficticio».
Resultado esperado	Se devuelve el objeto esperado con sus cinco campos y se registra una llamada a la red simulada.	El resultado carece de urgency_level y standardized_symptoms. Se reproduce la falta de comprobación de estos campos en el servicio.	Se obtiene urgencia «Medium» y se conserva «relato ficticio» como texto estandarizado de respaldo.
Resultado obtenido	Objeto igual al JSON simulado; una llamada al sustituto de red.	No se encontraron urgency_level ni standardized_symptoms.	Urgencia Medium y texto «relato ficticio».
Estado	Éxito	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito
Evidencia	triaje-python/resultado.json y salida.log; PU-40	triaje-python/resultado.json y salida.log; PU-41	triaje-python/resultado.json y salida.log; PU-42

Fuente: Elaboración propia, 2026

Tabla 19. Detalle de PU-43 a PU-45

Identificador	PU-43	PU-44	PU-45
Módulo	Triage mediante inteligencia artificial	Triage mediante inteligencia artificial	Videoconsultas
Caso de uso	Procesar y traducir sintomatología mediante IA	Procesar y traducir sintomatología mediante IA	Visualizar sala de espera
Función	transcribe_audio_only	transcribe_audio_only	isClosedTriage
Escenario	La transcripción simulada incluye espacios exteriores.	La solicitud de transcripción genera una excepción simulada.	El triaje tiene estado «completed».
Datos de prueba	Contenido ficticio codificado: ZmljdGljaW8=; respuesta « Texto transcrito ficticio ».	Contenido ficticio ZmljdGljaW8=; excepción «Fallo ficticio de red».	Estado completed.
Resultado esperado	Se obtiene «Texto transcrito ficticio» sin espacios exteriores. No se procesa una grabación real.	Se devuelve «[Error: No se pudo transcribir el audio adjunto]».	La función indica que el triaje está cerrado.
Resultado obtenido	Texto «Texto transcrito ficticio».	Mensaje «[Error: No se pudo transcribir el audio adjunto]».	true, triaje cerrado.
Estado	Éxito	Éxito	Éxito
Evidencia	traje-python/resultado.json y salida.log; PU-43	traje-python/resultado.json y salida.log; PU-44	videoconsultas/resultado.json y salida.log; PU-45

Fuente: Elaboración propia, 2026

Tabla 20. Detalle de PU-46 a PU-48

Identificador	PU-46	PU-47	PU-48
Módulo	Videoconsultas	Videoconsultas	Videoconsultas
Caso de uso	Visualizar sala de espera	Visualizar sala de espera	Visualizar sala de espera
Función	isClosedTriage	isClosedTriage	isDoctorReadyForCall
Escenario	El triaje tiene estado «resolved».	El triaje permanece en espera.	La atención está en progreso, sin una cita recibida.
Datos de prueba	Estado resolved.	Estado waiting.	Estado in_progress; cita nula.
Resultado esperado	La función indica que el triaje está cerrado.	La función indica que el triaje no está cerrado.	La decisión local permite habilitar la llamada; no se establece una videoconferencia.
Resultado obtenido	true, triaje cerrado.	false, triaje no cerrado.	true, habilitación local.
Estado	Éxito	Éxito	Éxito
Evidencia	videoconsultas/ resultado.json y salida.log; PU-46	videoconsultas/ resultado.json y salida.log; PU-47	videoconsultas/ resultado.json y salida.log; PU-48

Fuente: Elaboración propia, 2026

Tabla 21. Detalle de PU-49 a PU-51

Identificador	PU-49	PU-50	PU-51
Módulo	Videoconsultas	Videoconsultas	Videoconsultas
Caso de uso	Visualizar sala de espera	Visualizar sala de espera	Visualizar sala de espera
Función	isDoctorReadyForCall	isDoctorReadyForCall	isDoctorReadyForCall
Escenario	Hay un objeto de cita y el triaje sigue en espera.	No hay una cita ni atención en progreso.	El objeto de cita tiene estado cancelado.
Datos de prueba	Estado waiting; cita con id «cita-ficticia».	Estado waiting; cita nula.	Estado waiting; cita con status cancelled.
Resultado esperado	La decisión local permite habilitar la llamada.	La decisión local no habilita la llamada.	La decisión permite habilitar la llamada porque solo comprueba la existencia del objeto de cita.
Resultado obtenido	true, habilitación local.	false, sin habilitación local.	true, incluso con cita cancelada.
Estado	Éxito	Éxito	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional
Evidencia	videoconsultas/ resultado.json y salida.log; PU-49	videoconsultas/ resultado.json y salida.log; PU-50	videoconsultas/ resultado.json y salida.log; PU-51

Fuente: Elaboración propia, 2026

Tabla 22. Detalle de PU-52 a PU-54

Identificador	PU-52	PU-53	PU-54
Módulo	Historial y recetas	Historial y recetas	Historial y recetas
Caso de uso	Registrar historial clínico	Registrar historial clínico	Registrar historial clínico
Función	hasClinicalRecordFields	hasClinicalRecordFields	hasClinicalRecordFields
Escenario	El diagnóstico está vacío.	El tratamiento está vacío.	El diagnóstico y el tratamiento tienen contenido.
Datos de prueba	Diagnóstico vacío; tratamiento «Plan ficticio».	Diagnóstico «Diagnóstico ficticio»; tratamiento vacío.	«Diagnóstico ficticio» y «Plan ficticio».
Resultado esperado	La validación rechaza los campos.	La validación rechaza los campos.	Los campos superan la validación local. No se guarda un historial.
Resultado obtenido	false, campos rechazados.	false, campos rechazados.	true, campos aceptados localmente.
Estado	Éxito	Éxito	Éxito
Evidencia	historial-recetas/ resultado.json y salida.log; PU-52	historial-recetas/ resultado.json y salida.log; PU-53	historial-recetas/ resultado.json y salida.log; PU-54

Fuente: Elaboración propia, 2026

Tabla 23. Detalle de PU-55 a PU-57

Identificador	PU-55	PU-56	PU-57
Módulo	Historial y recetas	Historial y recetas	Historial y recetas
Caso de uso	Registrar historial clínico	Visualizar prediagnóstico de IA	Visualizar prediagnóstico de IA
Función	hasClinicalRecordFields	prepareTriageView	prepareTriageView
Escenario	El diagnóstico y el tratamiento contienen solo espacios.	Se recibe un reporte con nombre, urgencia y análisis.	Se recibe un objeto de triaje vacío.
Datos de prueba	Tres espacios en cada campo.	Paciente Ficticio; urgencia Low; análisis «Texto de prueba»; sin recomendación.	Objeto vacío.
Resultado esperado	Los campos superan la validación local. Se reproduce una limitación de la regla existente.	Se conservan el nombre, Low y «Texto de prueba»; la recomendación es «Sin recomendación.».	Se obtiene nombre «Desconocido» y urgencia «Medium» como valores predeterminados.
Resultado obtenido	true, se aceptaron los espacios.	Paciente Ficticio; análisis con Low, Texto de prueba y Sin recomendación.	Nombre Desconocido y urgencia Medium.
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional
Evidencia	historial-recetas/ resultado.json y salida.log; PU-55	historial-recetas/ resultado.json y salida.log; PU-56	historial-recetas/ resultado.json y salida.log; PU-57

Fuente: Elaboración propia, 2026

Tabla 24. Detalle de PU-58 a PU-60

Identificador	PU-58	PU-59	PU-60
Módulo	Historial y recetas	Historial y recetas	Historial y recetas
Caso de uso	Visualizar prediagnóstico de IA	Autogenerar receta PDF	Autogenerar receta PDF
Función	prepareTriageView	generatePrescriptionHtml	generatePrescriptionHtml
Escenario	No se recibe ningún triaje.	Se prepara el contenido de una receta sin farmacia.	Se incluye una farmacia en la receta.
Datos de prueba	Argumento sin definir.	Paciente Ficticio; Médico Ficticio; Diagnóstico ficticio; Plan ficticio; farmacia vacía.	Los mismos datos ficticios y farmacia «Farmacia Ficticia».
Resultado esperado	Se obtiene el nombre de demostración «Juan Perez» y urgencia «Critical». No corresponden a un paciente evaluado.	El HTML contiene paciente, médico, diagnóstico y tratamiento; no contiene «Farmacia Aliada Sugerida».	El HTML contiene «Farmacia Aliada Sugerida» y «Farmacia Ficticia». No se genera el archivo PDF.
Resultado obtenido	Nombre Juan Perez y urgencia Critical.	Los cuatro textos están presentes; la sección de farmacia está ausente.	Están presentes el título de farmacia y Farmacia Ficticia.
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito
Evidencia	historial-recetas/ resultado.json y salida.log; PU-58	historial-recetas/ resultado.json y salida.log; PU-59	historial-recetas/ resultado.json y salida.log; PU-60

Fuente: Elaboración propia, 2026

Tabla 25. Detalle de PU-61 a PU-63

Identificador	PU-61	PU-62	PU-63
Módulo	Pagos	Pagos	Pagos
Caso de uso	Verificar transacción	Verificar transacción	Verificar transacción
Función	markSimulatedPaymentWaiting	markSimulatedPaymentWaiting	markSimulatedPaymentWaiting
Escenario	La simulación de pago no recibe un identificador.	La simulación recibe un identificador ficticio.	Supabase simulado devuelve un objeto de error.
Datos de prueba	Identificador sin definir; Supabase sustituido.	Id «traje-ficticio»; respuesta simulada sin error.	Id «traje-ficticio»; error con mensaje «Fallo ficticio».
Resultado esperado	No se solicita una tabla ni una actualización.	Se solicita la tabla triages, el estado waiting y el filtro id igual a «traje-ficticio». No se realiza un pago.	La función termina sin devolver valor ni registrar el error recibido. No propaga el fallo.
Resultado obtenido	No se llamó al sustituto supabase.from.	Se comprobaron triages, actualización waiting y filtro id igual a triaje-ficticio.	Resolución undefined; ninguna llamada a console.error.
Estado	Éxito	Éxito	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional
Evidencia	pagos/resultado.json y salida.log; PU-61	pagos/resultado.json y salida.log; PU-62	pagos/resultado.json y salida.log; PU-63

Fuente: Elaboración propia, 2026

Tabla 26. Detalle de PU-64 a PU-66

Identificador	PU-64	PU-65	PU-66
Módulo	Pagos	Reportes	Reportes
Caso de uso	Verificar transacción	Generar reportes estadísticos	Generar reportes estadísticos
Función	markSimulatedPaymentWaiting	calculateReportStats	calculateReportStats
Escenario	Supabase simulado lanza una excepción.	No existen triajes en la colección.	La colección combina niveles conocidos y desconocidos.
Datos de prueba	Id «traje-ficticio»; excepción «Fallo ficticio».	Lista vacía.	Cuatro registros: Critical, Medium, Low y Unknown; sin estado.
Resultado esperado	La excepción se registra mediante console.error y la función termina sin propagarla.	Todos los contadores y la estimación de ingresos son cero.	Total: 4; un registro por cada nivel reconocido; el nivel desconocido no incrementa esas categorías. Estimación y completados: 0.
Resultado obtenido	Resolución undefined; console.error recibió el mensaje y la excepción esperados.	Los seis valores estadísticos son 0.	Total 4; Critical 1; Medium 1; Low 1; ingresos 0; completados 0.
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito
Evidencia	pagos/resultado.json y salida.log; PU-64	reportes/resultado.json y salida.log; PU-65	reportes/resultado.json y salida.log; PU-66

Fuente: Elaboración propia, 2026

Tabla 27. Detalle de PU-67 a PU-69

Identificador	PU-67	PU-68	PU-69
Módulo	Reportes	Reportes	Reportes
Caso de uso	Generar reportes estadísticos	Generar reportes estadísticos	Generar reportes estadísticos
Función	calculateReportStats	calculateReportStats	calculateReportStats
Escenario	Hay registros en espera y en progreso.	Hay un registro con estado completado.	Los estados no participan en la estimación existente.
Datos de prueba	Dos registros: waiting e in_progress.	Un registro con estado completed.	Dos registros: pending y resolved.
Resultado esperado	El campo completedTriages devuelve 2 y totalRevenue devuelve 100. Se reproduce el conteo de estados aún no finalizados.	Se obtiene 1 en completedTriages y 50 en totalRevenue. El importe es una estimación, no un cobro verificado.	completedTriages y totalRevenue son cero para estos estados.
Resultado obtenido	completedTriages=2; totalRevenue=100.	completedTriages=1; totalRevenue=50.	completedTriages=0; totalRevenue=0.
Estado	Éxito al reproducir el comportamiento actual; se detectó una limitación funcional	Éxito	Éxito
Evidencia	reportes/resultado.json y salida.log; PU-67	reportes/resultado.json y salida.log; PU-68	reportes/resultado.json y salida.log; PU-69

Fuente: Elaboración propia, 2026

Respaldo documental y evidencias gráficas

Documento oficial vigente: Documento_Oficial_V2 (5).docx, que sustituye al archivo anterior. La jerarquía existente ubica Pruebas unitarias en 3.3.1. No se encontró una secuencia de apéndices en el índice vigente, por lo que el extracto utiliza el título APÉNDICE sin copiar el número 3 del modelo. La numeración de tablas y figuras identifica el extracto independiente.

Referencias de presentación: PG - MODLEO_UML_RUP 2 1 (1).pdf, página impresa 147 para Objetivo, Acción y Efecto y página impresa 184 para las cinco columnas de casos; plantilla Demo_pruebas_unitarias_RUP2_documento_completo.docx. Los documentos de referencia permanecen sin sobrescribir.

Las capturas reales están pendientes. Sus espacios se identifican en Pruebas_unitarias_RUP2_resultados_actualizados.docx. La guía independiente Guía_capturas_pruebas_unitarias.md especifica el archivo, la parte visible, el propósito, el título y la fuente de cada figura. Los logs y JSON respaldan los resultados, pero no sustituyen las imágenes exigidas.

Los resultados finales muestran que las respuestas evaluadas coincidieron con las expectativas definidas. Los hallazgos de caracterización siguen pendientes, y las actividades de integración, las pruebas manuales y la evaluación clínica delimitan el trabajo aún necesario. No se concluye que el sistema esté libre de errores.